Technical Documentation

RAG-Based Customer Support Assistant

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines:

- Information retrieval (from documents)
- Text generation (using LLMs)

It improves accuracy by providing context to the model.

Why is RAG Needed?

Traditional LLMs:

- May give incorrect answers (hallucination)
- Lack domain-specific knowledge

RAG solves this by retrieving relevant data before generating responses.

Use Case Overview

This system acts as a **customer support assistant** that:

- Reads PDF documents
- Retrieves relevant information
- Generates accurate answers

2. System Architecture Explanation

The system follows a pipeline:

1. PDF ingestion
2. Text chunking
3. Embedding generation
4. Vector storage
5. Query processing
6. Retrieval of relevant chunks
7. LLM-based response generation

Component Interaction

- User → Query input
- Retriever → Fetch relevant chunks
- LLM → Generate answer
- LangGraph → Control workflow
- HITL → Handle complex queries

3. Design Decisions

Chunk Size Selection

- Chunk size: ~800 characters
- Reason: Balance between context and performance

Embedding Strategy

- Model: all-MiniLM-L6-v2
- Reason: Lightweight and efficient

Retrieval Approach

- Top-k similarity search
- Retrieves most relevant chunks

Prompt Design Logic

- Includes context + user query
- Ensures accurate responses

4. Workflow Explanation

The system uses **LangGraph** for workflow management:

Steps

1. User enters query
2. Query is processed
3. Relevant chunks are retrieved
4. LLM generates response
5. Output is displayed

Node Responsibilities

- Processing Node: Handles retrieval + LLM
- Output Node: Displays result

State Transitions

- Query → Processing → Output

5. Conditional Logic

Intent Detection

System checks query relevance and confidence score

Routing Decisions

- High confidence → Output
- Low confidence → HITL

6. HITL Implementation

Role of Human Intervention

Used when:

- Model confidence is low
- Context is insufficient

Process

1. System detects low confidence
2. Human provides answer
3. Response is returned to user

Benefits

- Improves reliability
- Reduces incorrect answers

Limitations

- Requires human availability
- Slower response time

7. Challenges & Trade-offs

Retrieval Accuracy vs Speed

- More chunks = better accuracy but slower

Chunk Size vs Context Quality

- Larger chunks = more context
- Smaller chunks = faster retrieval

Cost vs Performance

- API-based LLMs cost more
- Local models are slower

8. Testing Strategy

Testing Approach

- Functional testing
- Edge case testing
- Performance testing

Sample Queries

- What is machine learning?
- Explain artificial intelligence
- Difference between ML and AI

9. Future Enhancements

- Multi-document support
- Chat memory integration
- Web-based UI
- Deployment on cloud
- Feedback learning system

Conclusion

This project demonstrates the design and implementation of a scalable RAG-based system. It improves response accuracy by combining retrieval and generation, making it suitable for real-world customer support applications.